Webpack 原理解析 - 编译准备阶段

编译准备阶段（2-1）

在得到最终的配置参数之后，Webpack在开始编译前做了以下几件事情:

1. 通过参数创建 `compiler` 对象。我们可以看到webpack源码中通过调用 `webpack(options)` 方法返回的是一个 `compiler` 对象。并且同时调用 `compiler.run()` 方法启动的代码进行打包
2. 注册我们定义的 `webpack plugin` 插件。
3. 根据传入的配置对象寻找对应的打包入口文件。

完善 `index.js` 中的逻辑代码

```
// core/index.js
const webpack = require('./webpack.js')
const config = require('../webpack.config.js')

// 1: 初始化参数 将配置文件和shell参数合成最终配置对象
// 2: 调用Webpack(options) 初始化compiler对象
// 3: webpack()方法返回一个compiler对象
const compiler = webpack(config)
```

完善 `webpack()` 方法

```
// core/webpack.js
const Compiler = require('./compiler.js')

const webpack = (options) => {
  // 1: 合并参数 得到合并后的参数对象 mergeOptions
  const mergeOptions = getNormalizedWebpackOptions(options)
  console.log('chaser: webpack -> mergeOptions', mergeOptions)

  // 2: 调用Webpack(options) 初始化compiler对象
  const compiler = new Compiler(mergeOptions)
  // 3: 执行compiler.run()方法进行打包
  compiler.run((err, stats) => {
    if (err) {
```

```
        console.log('chaser: compiler', err)
      }
    })
    return compiler
  }

  module.exports = webpack
```

```
// core/compiler.js

// Compiler类进行核心编译实现
class Compiler {
  constructor(options) {
    this.options = options
  }

  /**
   * run方法启动编译
   * @param {*} callback 运行结束后触发的回调
   */
  run(callback) {}
}

module.exports = Compiler
```

此时我们的 `Compiler` 类已经搭建好一个基础的骨架代码。

- `core/index.js` 作为打包命令的入口文件，这个文件引用了我们自己实现的 `webpack` 同时引用了外部的 `webpack.config.js(options)`
- `core/webpack.js` 这个文件目前处理了参数的合并以及传入合并后的参数 `new Compiler(mergeOptions)` ，调用 `webpack(options).run()` 开始编译，同时返回 `Compiler` 实例对象
- `core/compiler.js` ，此时我们的 `compiler` 仅仅是作为一个基础的骨架，只存在一个 `run()` 启动方法

编写 `plugin` 插件（2-2）

完善 `Compiler` 方法

```
const { SyncHook } = require('tapable')

// Compiler类进行核心编译实现
class Compiler {
  constructor(options) {
    this.options = options
    // 创造plugin hooks
    this.hooks = {
      // 开始编译时的钩子（调用run方法后）
      run: new SyncHook(),
      // 输出asset文件前的钩子（写入文件之前）
      emit: new SyncHook(),
      // 完成编译时的钩子（compilation执行完成后）
      done: new SyncHook()
    }
  }

  /**
   * run方法启动编译
   * @param {*} callback 运行结束后触发的回调
   */
  run(callback) {
    // 调用run方式 触发开始编译的plugin钩子
    this.hooks.run.call()
  }
}

module.exports = Compiler
```

我们在 `Compiler` 这个类的构造函数中创建了一个属性 `hooks` ，它的值是三个属性 `run` 、 `emit` 、 `done` 。利用 `Tapable` 为这些属性创建一系列钩子，本质上你可以简单将 `SyncHook()` 方法理解称为一个 `Emitter Event` 类。

compiler 钩子

Tapable文档

```
// tapable 使用方式
const { SyncHook } = require('tapable')

class Test {
  constructor() {
    this.hooks = {
      done: new SyncHook()
    }
  }
```

```

const myTest = new Test()

// 注册钩子
myTest.hook.done.tap('name', callback1)
myTest.hook.done.tap('name', callback2)

// 执行所有通过top注册的钩子
myTest.hook.done.call()
```

`plugin`

```
// plugin的本质是一个带有apply方法的对象
// apply函数会在初始化阶段，创建好Compiler对象后运行。
class MyPlugin{
    apply(compiler){
        compiler.hooks.事件名称.事件类型(name, function(compilation){
            //事件处理函数
        })
    }
}
```

💡 事件名称：即要监听的事件名

事件类型：
tap：注册一个同步的钩子函数，函数运行完毕则表示事件处理结束。
tapAsync：注册一个基于回调的异步的钩子函数，函数通过调用一个回调表示事件处理结束。
tapPromise：注册一个基于Promise的异步的钩子函数，函数通过返回的Promise进入已决状态表示事件处理结束。

完善 `webpack()` 方法

```
// core/webpack.js
const Compiler = require('./compiler.js')

const webpack = (options) => {
  // 1: 合并参数 得到合并后的参数对象 mergeOptions
  const mergeOptions = getNormalizedWebpackOptions(options)
```

```
  // 2: 调用Webpack(options) 初始化compiler对象
  const compiler = new Compiler(mergeOptions)

  // 3: 注册webpack plugin钩子
  if (Array.isArray(plugins)) {
    options.plugins.forEach((plugin) => {
      plugin.apple(compiler)
    })
  }

  // 4: 执行compiler.run()方法进行打包
  compiler.run((err, stats) => {
    if (err) {
      console.log('chaser: compiler', err)
    }
  })
  return compiler
}

module.exports = webpack
```

📌 `webpack` 插件本质上就是通过发布订阅的模式，在 `compiler` 上监听事件。然后再打包编译过程中触发监听的事件从而添加一定的逻辑影响打包结果。

配置 `entry` 入口（2-3）

💡 `webpack` 的每次打包都需要从入口文件开始，源码中通过 `EntryOptionPlugin` 插件处理 entry 配置。

```
// core/compiler.js
const { SyncHook } = require('tapable')
const path = require('path')

// Compiler类进行核心编译实现
class Compiler {
  constructor(options) {
    this.options = options
    // 创造plugin hooks
```

```
10      this.hooks = {
11        // 开始编译时的钩子（调用run方法后）
12        run: new SyncHook(),
13        // 输出asset文件前的钩子（写入文件之前）
14        emit: new SyncHook(),
15        // 完成编译时的钩子（compilation执行完成后）
16        done: new SyncHook()
17      }
18    }
19
20    /**
21     * run方法启动编译
22     * @param {*} callback 运行结束后触发的回调
23     */
24    run(callback) {
25      // 调用run方式 触发开始编译的plugin钩子
26      this.hooks.run.call()
27
28      // 获取入口路径
29      const entry = this.getEntry()
30      console.log('chaser: Compiler -> run -> entry', entry)
31    }
32
33    /**
34     * 获取入口文件路径
35     */
36    getEntry() {
37      // 转换成键值对形式
38      let entry = Object.create(null)
39      const { entry: optionEntry } = this.options
40      if (typeof optionEntry === 'string') {
41        entry['main'] = optionEntry
42      } else {
43        entry = optionEntry
44      }
45      // 转换绝对路径
46      Object.keys(entry).forEach((key) => {
47        const value = entry[key]
48        if (!path.isAbsolute(value)) {
49          entry[key] = path.join(process.cwd(), value)
50        }
51      })
52      return entry
53    }
54  }
55
56  module.exports = Compiler
```

```
// core/webpack.js
const Compiler = require('./compiler.js')

const webpack = (options) => {
  // 1: 合并参数 得到合并后的参数对象 mergeOptions

  // 2: 调用Webpack(options) 初始化compiler对象

  // 3: 注册webpack plugin钩子
  if (Array.isArray(plugins)) {
    options.plugins.forEach((plugin) => {
      plugin.apple(compiler)
    })
  }
  // 处理入口文件 源码中该过程由EntryOptionPlugin插件完成
  compiler.getEntry()

  // 4: 执行compiler.run()方法进行打包
}

module.exports = webpack
```

到这里，`compiler` 实例就被创建好了，相应的环境参数也预设好了，紧接着开始调用 `compiler.compile` 函数开始编译代码。

END